Task 1: The Crash Gallery

Background

This task is about breaking code and observing what happens. You will be given a set of small Python functions that work on the happy path but fail on bad input. Your job is to call each function with bad input, record the exact error, and then patch the function so it fails gracefully instead of crashing.

The goal is not to write perfect code yet. The goal is to see how many ways code can break and to develop intuition for where problems hide.

What to Do

Part A: The Broken Functions

Create a file called `crash_gallery.py`. Copy the following functions into it exactly as written. Do not fix them yet.

python

```python
def divide(a, b):
    return a / b

def get_first(items):
    return items[0]

def get_age(person):
    return person["age"]

def parse_number(text):
    return int(text)

def read_file(filename):
    file = open(filename, "r")
    content = file.read()
    file.close()
```

```
    return content

def average(numbers):
    return sum(numbers) / len(numbers)
```

Part B: Break Each Function

Write a small test block at the bottom of the file that calls each function with bad input. For each function, try at least three different kinds of bad input. Record what happens in a table like the one below.

Function	Bad Input	What Happened	Error Type
divide	divide(10, 0)		
divide	divide("10", 2)		
divide	divide(10, None)		
get_first	get_first([])		
get_first	get_first(None)		
get_age	get_age({})		
get_age	get_age("Ali")		
parse_number	parse_number("abc")		
parse_number	parse_number(None)		
read_file	read_file("missing.txt")		
average	average([])		
average	average(["a", "b"])		

Part C: Patch Each Function

Now rewrite each function so that it does not crash on bad input. Instead of crashing, it should return a clear error message or a safe default. Do not use `try` and `except` yet if you have not learned it. Use `if` checks to guard against bad input.

For example, the first function could be rewritten like this.

```python
def divide(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        return "Error: both arguments must be numbers"
    if b == 0:
        return "Error: cannot divide by zero"
    return a / b
```

Rewrite all six functions in this style. Call each one with the same bad inputs and confirm that none of them crash.

Part D: Discussion Notes

Write short answers to these questions in your report. Which function was hardest to make safe, and why. Which kind of bad input was most common across the functions. What pattern did you notice in how you fixed each function. Did any function need more than one check, and why.

Expected Output : Your patched functions should produce output like this when called with bad input.

```text
divide(10, 0) -> Error: cannot divide by zero
divide("10", 2) -> Error: both arguments must be numbers
get_first([]) -> Error: list is empty
get_first(None) -> Error: input is not a list
get_age({}) -> Error: missing age
parse_number("abc") -> Error: not a number
read_file("missing.txt") -> Error: file not found
average([]) -> Error: cannot average an empty list
```

The exact messages can differ. The important thing is that nothing crashes.

Task 2: The Silent Bug Hunt

Not all bugs cause crashes. Some bugs cause silent wrong results, which are far more dangerous because nobody notices them until much later. This task is about finding silent bugs, which means code that runs without errors but produces wrong answers.

You will be given a small program that looks correct but has several silent bugs. Your job is to find them, explain why they are wrong, and fix them. This is exactly what defensive programming is about. Defensive code does not just avoid crashes. It also avoids silently wrong results.

What to Do

Part A: The Buggy Program

Create a file called `bug_hunt.py`. Copy the following program into it.

python

```python
def calculate_discount(price, discount_percent):
    return price - price * discount_percent / 100

def calculate_average(marks):
    total = 0
    for mark in marks:
        total = total + mark
    return total / len(marks)

def find_student(students, name):
    for student in students:
        if student["name"] == name:
            return student
    return students[0]

def is_adult(age):
    if age > 18:
        return True
    else:
        return False
```

```python
def apply_tax(amount, tax_rate):
    return amount + amount * tax_rate

def count_passed(marks, passing_mark=50):
    count = 0
    for mark in marks:
        if mark > passing_mark:
            count = count + 1
    return count
```

Part B: Find the Silent Bugs

For each function, write at least three test cases that include edge cases. An edge case is a value at the boundary of what is allowed, such as zero, negative numbers, empty lists, or exact threshold values. Run each test and record the result.

Function	Test Input	Expected Result	Actual Result	Bug?
calculate_discount	(100, 0)	100		
calculate_discount	(100, 150)	error or capped		
calculate_average	[]	error		
find_student	missing name	None or error		
is_adult	18	True (18 is adult)		
is_adult	17	False		
apply_tax	(100, -0.1)	error		
count_passed	[50, 50, 50]	3		

Part C: Fix the Bugs

Rewrite each function so that it handles the edge cases correctly and produces clear results. For each fix, add a comment explaining what the original bug was and how the fix addresses it.

Here is an example of what a fixed function looks like.

python

```python
def is_adult(age):
    # Bug: original used > 18, which excludes 18 year olds
    # Fix: use >= 18 so that 18 year olds are counted as adults
    if not isinstance(age, int):
        return False
    if age < 0:
        return False
    return age >= 18
```

Part D: The Bug Report

Write a short bug report in your lab document. For each bug you found, write three sentences. What the function was supposed to do. What it actually did. What you changed to fix it. Then answer these questions. Which bug was the hardest to find, and why. How did writing edge case tests help you find bugs that normal tests would miss. Why is a silent wrong result more dangerous than a crash.

Expected Output

When you run your fixed functions with the edge case inputs, the output should be correct and clear.

text

```
calculate_discount(100, 0) -> 100
calculate_discount(100, 150) -> Error: discount must be between 0 and 100
calculate_average([]) -> Error: cannot average empty list
find_student(students, "missing") -> None
is_adult(18) -> True
is_adult(17) -> False
apply_tax(100, -0.1) -> Error: tax rate cannot be negative
count_passed([50, 50, 50]) -> 3
```